Naive Bayes and Bayesian Networks

ES 335 Machine Learning, Semester I 2025–26 Sources: lectures "Naive Bayes" and "Bayesian networks", cheat sheet ML27

What this file is really about:

$$\text{Bayes' rule} \rightarrow \text{independence given the class} \rightarrow \text{count} \rightarrow \text{smooth} \rightarrow \text{add logs} \rightarrow \text{graphs of depe}$$

How to use this file. Part A works on small tables you can count by hand. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Conditional probability and the normal density.

1. Turn a test result around with Bayes' rule.
2. Assume features are independent given the class.
3. Fit by counting, with smoothing.
4. Compute in logs, and use a normal per feature.
5. Draw dependence as a graph.

You can stop when you can classify an email by hand and explain why adding logs is safe.

Contents

Part A Maths From the Ground Up

A1 Bayes' rule turns a test around

A test is right 99% of the time for the sick and for the healthy. One person in 10,000 has the disease. If you test positive, how likely are you to be sick?

Among a million people, 100 are sick and 99 test positive. Of the 999,900 healthy, 1% also test positive, which is 9,999 people. So

$$P(\text{sick} \mid +) = \frac{99}{99 + 9999} \approx 0.0098.$$

The rule behind it is

$$P(y \mid x) = \frac{P(x \mid y)\,P(y)}{P(x)}, \qquad P(x) = \sum_{c} P(x \mid c) P(c).$$

A2 Why a naive assumption

To classify, we need $P(x \mid y)$. With n binary features the full table has 2^n rows per class, far too many to estimate.

Naive Bayes assumes the features are independent once the class is known:

$$\hat{y} = \arg\max_{y} P(y) \prod_{j=1}^{n} P(x_j \mid y).$$

$P(x)$ is the same for every class, so it can be dropped. With 30 binary features this needs 61 numbers instead of about 2×10^9. The assumption is rarely true, yet the ranking of classes is often right.

A3 Fitting is counting

Ten emails, each with three words present or absent, half spam and half ham:

	w_1	w_2	w_3
$P(w_j = 1 \mid \text{spam})$	2/5	2/5	3/5
$P(w_j = 1 \mid \text{ham})$	3/5	3/5	2/5

Maximum likelihood gives simply the frequency within each class, and the prior is the class's share of the rows.

Worked example. For the email $(0, 0, 1)$, spam scores $(3/5)^3 \times 1/2 = 0.108$ and ham scores $(2/5)^3 \times 1/2 = 0.032$. Normalising gives $P(\text{spam}) = 0.108 / 0.140 = 0.771$.

A4 One zero vetoes everything

Careful. If a word never appeared in spam, its probability is 0, so the product is 0 whatever the other words say. **Laplace smoothing** adds a small count to every cell:

$$P(x_j = 1 \mid y = c) = \frac{\text{count}(x_j = 1, y = c) + \alpha}{\text{count}(y = c) + 2\alpha}.$$

With $\alpha = 1$ on the ten emails, $(0, 0, 1)$ gets 0.703.

A5 Add logs, do not multiply

Hundreds of factors below 1 underflow to zero, and then every class ties. Taking logs keeps the same winner:

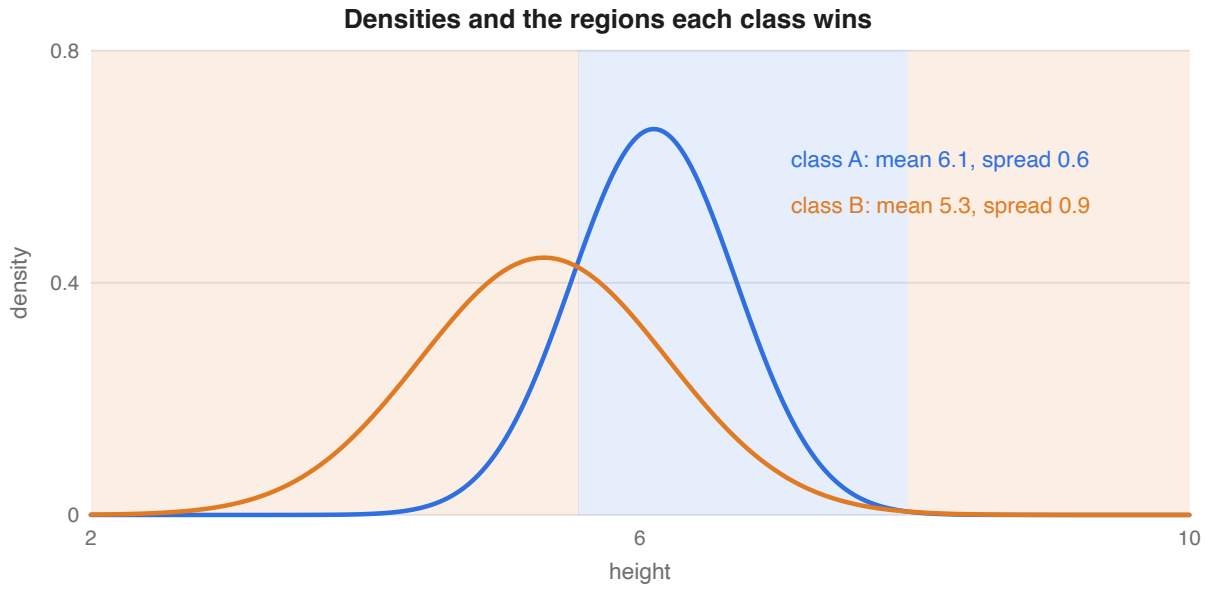
$$\hat{y} = \arg\max_y \Big[\log P(y) + \sum_j \log P(x_j \mid y) \Big].$$

A6 Real-valued features

For a continuous feature, fit one normal per class, using the mean and variance of that class's rows:

$$p(x \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\Big(-\frac{(x-\mu_k)^2}{2\sigma_k^2} \Big).$$

Densities can exceed 1: compare them, but do not read them as probabilities. If the classes have different variances, the boundary is quadratic and can have two cut points.



Heights for two classes with different spreads. The wider class wins in both tails, so the boundary has two cut points.

A7 Generative against discriminative

Naive Bayes models $P(x \mid y)P(y)$, while logistic regression models $P(y \mid x)$ directly. With variances shared by the classes, Gaussian naive Bayes also gives linear log-odds, but its weights are set by counting rather than fitted. Naive Bayes needs less data. With more data and correlated features, logistic regression usually wins.

A8 Bayesian networks

A **Bayesian network** draws variables as nodes and direct influence as arrows, with no directed cycles. The joint distribution factorises along the graph:

$$P(X_1, \ldots, X_N) = \prod_k P(X_k \mid \text{parents}(X_k)).$$

Naive Bayes is the graph where the class points at each feature. A node with k binary parents needs 2^k numbers.

Explaining away. Suppose theft and an earthquake can each set off an alarm. Hearing the alarm raises the probability of theft. Then learning there was an earthquake lowers it again, because the alarm has already been explained.

Each independent positive test multiplies the odds by $0.99/0.01 = 99$, taking the disease from a probability of 0.0098 after one test to about 0.495 after two.

Part B In Code

B1 Counting with smoothing

```
theta = (X[y == c].sum(0) + alpha) / ((y == c).sum() + 2 * alpha)
```

B2 Scoring in log space

```
log_p = np.log(prior[c]) + (x * np.log(theta) + (1 - x) * np.log(1 - theta)).sum()
```

B3 Gaussian naive Bayes

```
mu, var = X[y == c].mean(0), X[y == c].var(0, ddof=1)
log_p = -0.5 * (np.log(2 * np.pi * var) + (x - mu) ** 2 / var).sum()
```

Part C Practice

P1. A disease affects 1 in 1000 people, and a test has sensitivity and specificity of 90%. Find $P(\text{sick} \mid +)$.

$$\frac{0.9 \times 0.001}{0.9 \times 0.001 + 0.1 \times 0.999} = 0.0089.$$

P2. With 20 binary features and two classes, how many numbers does naive Bayes need?

$2 \times 20 + 1 = 41$.

P3. A word appears in 0 of 8 spam emails. Find its smoothed probability with $\alpha = 1$.

$(0 + 1)/(8 + 2) = 0.1$.

P4. All four overcast days in a table were "yes". Without smoothing, what does naive Bayes give "no" for an overcast day?

Probability 0, whatever the other features say.

P5. Why compute with logs?

Long products underflow to zero and every class ties. A sum of logs keeps the same winner.

P6. How many free numbers does a node with three binary parents need?

$2^3 = 8$.